Access Control System and Surveillance with Web Cam

Laboratory of Internet of Things

Jacopo Veronese, 257771

***Abstract*—*This report describes the design and implementation of a QR-code-based access control and surveillance system for a shared laboratory environment, built on a Raspberry Pi. A short-lived, cryptographically signed QR code, generated after PIN authentication, grants time-boxed access to a zone; a camera-based scanner validates the code, logs the event to a time-series database, notifies an operator over Telegram, and can optionally actuate a physical door strike through a GPIO relay. All device coordination uses MQTT as the primary communication protocol, and an access control list can be managed either from a command-line interface or from a lightweight web front end. The system was implemented in modular Python and evaluated component-by-component on real Raspberry Pi hardware, with MQTT messaging, time-series persistence, QR generation/validation, and the web management interface all confirmed working end-to-end; camera-based scanning was implemented and integration-tested but could not be fully validated due to a hardware fault discovered late in development.*

I. INTRODUCTION

Access control in shared spaces such as university laboratories is traditionally handled with physical keys or static RFID badges, both of which are inconvenient to provision, revoke, and audit. A badge lost or shared cannot easily be traced, and adding or removing a person's access typically requires physical presence at an administration desk. The Internet of Things (IoT) offers a more flexible alternative: an embedded controller can authenticate users through a short-lived, digitally signed credential displayed on a personal device, verify it optically with a camera, and immediately log, notify, and — where wired to real hardware — physically actuate the door, all while access rights are managed remotely.

This project implements such a system on a Raspberry Pi, following the specification of Project 7 ("Access Control System and Surveillance with Web Cam"): QR-code authentication, a Raspberry Pi camera as the scanning sensor, MQTT as the inter-component protocol, optional Telegram notifications, a user interface for managing the access list, and optional hardware actuation of a door lock.

The main contributions of this work are:

- A signed, time-limited QR-code authentication scheme (HMAC-SHA256 over a JSON payload) that prevents replay beyond a short validity window without requiring a network round-trip to validate the signature itself.
- A modular Python implementation separating persistence, authentication, QR handling, camera scanning, MQTT messaging, notification, and door actuation into independent, individually testable components.
- A full MQTT request/response API covering authentication, QR generation and validation, and access-list management (add, remove, list), enabling third-party or future front ends to drive the system without touching its internals.
- A minimal web management interface, built as a complement to the command-line tool, that satisfies the requirement for a user-facing way to grant and revoke access.
- An honest, component-level evaluation of what was verified to work on real hardware versus what remains simulated or untested, discussed in Section VII.

The rest of the report is organized as follows. Section II reviews related approaches to IoT-based access control. Section III describes the system architecture. Section IV details the design rationale behind the main components. Section V presents the implementation, with representative code listings. Section VI describes the MQTT and web interfaces. Section VII reports the evaluation performed on the physical deployment. Section VIII discusses limitations and future work. Section IX concludes the report.

II. RELATED WORK

QR codes are widely used as a low-cost alternative to RFID or NFC for identity verification, since they require no specialized reader hardware beyond a camera and a decoding library such as ZBar [1]. Static QR codes used for access control are, however, vulnerable to photographing and replay; several commercial systems address this by regenerating the code periodically or embedding a signed, time-limited token, an approach this project adopts directly by combining a short validity window with an HMAC signature [2].

MQTT [3] is the de facto standard publish/subscribe protocol for constrained IoT devices, owing to its small header overhead and broker-mediated decoupling between publishers and subscribers; it is used here as the backbone connecting the access controller to any number of management or monitoring clients without those clients needing direct network access to the Raspberry Pi. For time-series logging of access events, purpose-built time-series databases such as InfluxDB [4] offer more natural query semantics (e.g., "most recent active record per user") than a general-purpose relational store, at the cost of requiring care when a single logical entity — a user's access grant — is represented as an evolving series of immutable points rather

than as an updatable row, a design consideration discussed further in Section V.

III. SYSTEM ARCHITECTURE

The system follows the modular architecture shown in Fig. 1. A user presents a signed QR code, generated on their own device after PIN authentication, to a Raspberry Pi camera. The decoded payload is validated by the orchestrator, which coordinates four supporting components: an Authenticator (PIN and time-window checks), a QR Manager (signing and verification), a persistence layer backed by InfluxDB, and a notification/actuation layer (Telegram and GPIO). All state-changing operations are additionally exposed over MQTT, and a Flask-based web UI provides a human-facing view onto the same persistence layer for managing the access list.

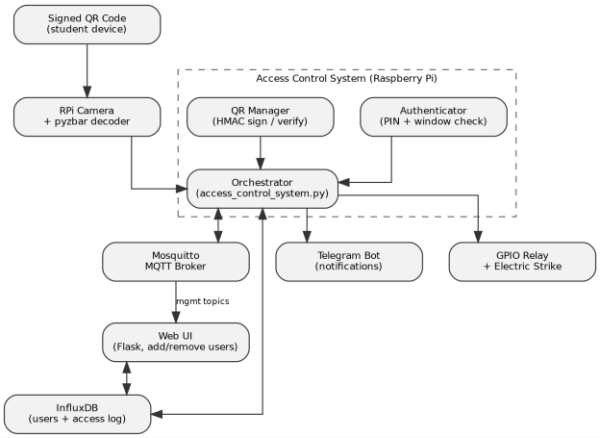

Fig. 1. System architecture and data flow.

A. Hardware

The prototype runs on a Raspberry Pi (Raspberry Pi OS, Debian 13 "trixie") with a Camera Module attached via the CSI connector for QR scanning, and is designed to drive a relay module wired to an electric door strike for physical actuation. Mosquitto acts as the local MQTT broker and InfluxDB 2.9 as the local time-series store; both run as systemd services on the same device, so the controller has no hard dependency on external network connectivity beyond delivering Telegram notifications.

B. Software Modules

- Database (database.py): all InfluxDB reads/writes — adding a user, revoking a user, listing active users, and logging access events.
- Authenticator (authenticator.py): verifies a submitted PIN against the stored record, checks the requested access window, and enforces a lockout after repeated failed attempts.
- QR Manager (qr_manager.py): generates a JSON payload (user, zone, expiry, unique ID), signs it with HMAC-SHA256 under a server-side master secret, and verifies incoming codes against that signature and a short validity window.
- Camera Scanner (camera_scanner.py): continuously captures frames via Picamera2, decodes QR codes with pyzbar, and — as a fallback for development without camera hardware — accepts pasted QR payloads from the terminal.
- Notifier (notifier.py): sends Telegram messages and photos via the Bot API.
- MQTT Handler (mqtt_handler.py): subscribes to a fixed set of request topics and publishes matching responses and broadcast events.
- Door Lock (gpio_lock.py): pulses a GPIO pin to actuate a relay for a configurable duration, with a simulated fallback when no relay is enabled.
- Web UI (web_app.py): a small Flask application exposing the access list as a browsable, editable table.

IV. DESIGN RATIONALE

A. Time-limited, signed QR codes

A QR code that never changes is equivalent to a photographable key: anyone who captures an image of it can replay it indefinitely. The system instead generates a fresh code on demand, valid for a short, configurable window (15 seconds by default), and signs its JSON payload with HMAC-SHA256 under a secret known only to the controller. A scanner can therefore reject a tampered payload without any database lookup — the signature alone proves the payload has not been altered — while the short validity window bounds how long a captured code remains useful even if intercepted in transit.

B. Append-only persistence and revocation

InfluxDB stores each user grant as an immutable, timestamped point rather than an updatable row. Revoking a user is therefore implemented by writing a new point with the same identifying tags and an active=false field, rather than deleting or mutating the original record — preserving a complete audit trail of every grant and revocation. This design choice has a direct consequence for lookups: if a user is re-added under a different zone, the two grants are held in separate series (Influx groups points by their full tag set), so a correct "is this user currently active" query must scan every series for that user and take the most recent point overall, not merely the most recent point within an arbitrarily chosen series — an issue encountered and corrected during testing, discussed in Section VII.

C. MQTT as the integration boundary

Rather than allowing the web UI or any future client to modify the database directly, every access-list operation is also exposed as an MQTT request/response pair (Table I). This keeps a single, well-defined boundary between "the system" and anything that wants to observe or drive it, so that a future graphical dashboard, a mobile app, or an automated provisioning script could be added without modifying the controller itself.

D. Fail-safe hardware integration

Both the camera and the GPIO door-lock modules detect at start-up whether the underlying hardware/library is actually usable, and fall back to a clearly-logged simulated or manual-input mode rather than crashing the whole controller. This allowed the majority of the system — authentication, persistence, MQTT, and the web UI — to be developed, tested, and demonstrated even while the physical camera module was non-functional (Section VII-C), and allows door-lock actuation to be exercised in software before any relay is physically wired.

V. IMPLEMENTATION

The implementation is organized as one Python module per responsibility (Section III-B), coordinated by a single orchestrator, access_control_system.py, instantiated once at start-up and shared by both the terminal menu and the MQTT handler.

A. QR signing and verification

```
def _sign(self, qr_data):
    qr_json = json.dumps(qr_data,
        separators=(',', ':'))
    return hmac.new(
        config.MASTER_SECRET.encode(),
        qr_json.encode(),
        hashlib.sha256
    ).hexdigest()
```

Listing 1. HMAC signing of the QR payload (qr_manager.py).

Validation recomputes the same signature over the received payload and compares it using a constant-time comparison (hmac.compare_digest), then checks the code against a table of currently outstanding, unexpired codes before consuming it — a code can only ever be validated once, closing the obvious replay window that a purely time-based check alone would leave open between generation and first use.

B. Access-window authentication

```
def authenticate(self, user_id, pin_code):
    if self._is_locked_out(user_id):
        return False, "Account locked"
    user_data = self.db.get_user_data(
        user_id)
    if not user_data:
        self._record_failed_attempt(user_id)
        return False, "User not found"
    if not self._verify_pin(
            user_data, pin_code):
        self._record_failed_attempt(user_id)
        return False, "Invalid PIN"
    if not self._is_window_valid(user_data):
        return False, "Access window"
                      " not valid"
    return True, user_data
```

Listing 2. PIN and time-window authentication (authenticator.py).

C. Revocation as an append-only write

```
def remove_user(self, user_id):
    existing = self._get_user_data(
        user_id)
    if not existing:
        return False
    point = (Point("authorized_users")
        .tag("user_id",
             existing["user_id"])
        .tag("zone", existing["zone"])
        .tag("pin", existing["pin"])
        .field("active", False)
        .time(datetime.utcnow(),
              WritePrecision.NS))
    self.write_api.write(
        bucket=config.INFLUX_BUCKET,
        org=config.INFLUX_ORG,
        record=point)
    return True
```

Listing 3. Revocation by writing an active=False point (database.py).

D. Door-lock actuation with simulated fallback

```
def unlock(self):
    if not self.ready:
        print(f"[door-lock] simulated "
              f"unlock for "
              f"{self.unlock_seconds}s")
        return
    def _pulse():
        with self._lock:
            GPIO.output(
                self.pin, GPIO.HIGH)
            time.sleep(
                self.unlock_seconds)
            GPIO.output(
                self.pin, GPIO.LOW)
    threading.Thread(
        target=_pulse,
        daemon=True).start()
```

Listing 4. GPIO relay pulse with a safe simulated fallback (gpio_lock.py).

VI. MQTT AND WEB INTERFACES

Table I summarizes the MQTT request/response API exposed by the controller. Every topic pair follows the same convention: a client publishes a JSON request carrying a request_id, and the controller republishes a JSON response on the matching *_response topic carrying the same request_id, so that a client issuing several concurrent requests can correlate responses without additional bookkeeping.

Operation	Request topic
Authenticate	`access/auth/request`
Generate QR	`access/qr/generate/request`
Validate QR	`access/qr/validate/request`
Add user	`access/user/add/request`
Remove user	`access/user/remove/request`
List users	`access/user/list/request`

*TABLE I. MQTT REQUEST TOPICS (each has a matching *_response topic)*

The web UI (Fig. 2) reads and writes the same InfluxDB-backed access list directly through the Database module, independent of the MQTT path, so that it can run without also starting the camera scanner or the MQTT client — a deliberate separation that let the two be developed and demonstrated independently.

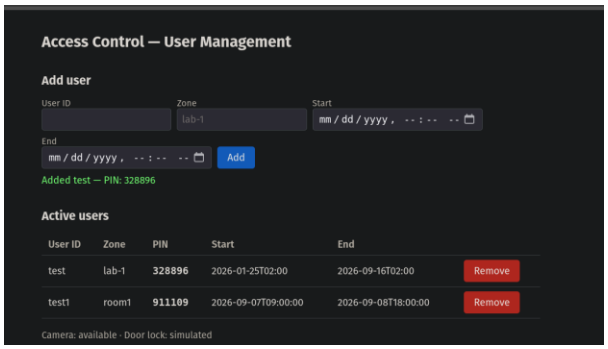

Fig. 2. Web management UI running on the deployed Raspberry Pi, showing a user added with its generated PIN.

A. Web API

The web front end exposes a small REST surface (GET/POST /api/users, DELETE /api/users/<id>, GET /api/log, GET /api/status) consumed by a single-page vanilla-JavaScript client embedded directly in the Flask response, avoiding a separate build step for what is intentionally a minimal interface.

```
@app.route("/api/users", methods=["POST"])
def api_add_user():
    data = request.get_json(force=True)
    user_id = (data.get("user_id")
               or "").strip()
    zone = (data.get("zone")
            or "").strip()
    check_in = data.get("check_in")
    check_out = data.get("check_out")
    if not all([user_id, zone,
                check_in, check_out]):
        return jsonify(
            {"error": "missing fields"}
        ), 400
    pin = f"{secrets.randbelow(
        900000) + 100000}"
    db.add_user(user_id, zone, pin,
                check_in, check_out)
    return jsonify(
        {"user_id": user_id, "pin": pin})
```

Listing 5. Add-user REST endpoint (web_app.py).

VII. SECURITY CONSIDERATIONS

Three deliberate security decisions shape the implementation, and one security lapse was discovered and corrected during the project — reported here for completeness rather than omitted.

A. Secrets management

All secrets — the InfluxDB API token, the HMAC master secret, and the Telegram bot token — are read from environment variables via a local, git-ignored .env file (config.py, using python-dotenv), rather than being hardcoded in source. config.validate_config() fails fast with a descriptive error at start-up if a required secret is missing, rather than surfacing a confusing failure later. An early prototype of this project did hardcode these three secrets directly in source and committed them to a public repository; this was identified during a code review pass, the secrets were removed from the codebase and rotated, and the configuration was migrated to the scheme described here. This is noted explicitly as a concrete example of the class of mistake the current architecture is designed to prevent structurally, not merely by convention.

B. Constant-time signature comparison

QR signature verification uses hmac.compare_digest rather than the built-in string equality operator. A naive comparison short-circuits on the first mismatched byte, which — while not the primary threat model for a QR code that must also be optically captured — is avoided on principle, since it costs nothing and removes an entire class of timing side-channel from the verification path.

C. Brute-force lockout

The Authenticator locks an account for a configurable duration after a configurable number of consecutive failed PIN attempts (three attempts, ten minutes, by default), bounding the effectiveness of an online guessing attack against the six-digit PIN space without requiring any external rate-limiting infrastructure.

VIII. DEVELOPMENT PROCESS

The project began as a single monolithic script combining QR handling, MQTT, InfluxDB access, camera control, and a text-based menu in one file. Before hardware testing began, this was refactored into the module boundaries described in Section III-B, on the reasoning that a system with this many independently-testable concerns (persistence, cryptographic signing, hardware I/O, networking) benefits more from clear module boundaries than from the convenience of a single file, particularly once a second interface (the web UI) needed to reuse the same persistence logic without dragging in MQTT or camera dependencies.

Testing then proceeded incrementally against the physical Raspberry Pi rather than against a mocked environment: the MQTT broker and InfluxDB were installed and brought up first, followed by the authentication and QR logic (verified through the manual paste-in test mode described in Section IX-A), then the web UI, with camera integration attempted last. This ordering surfaced the InfluxDB multi-series lookup bug (Section IX-B) early, while the rest of the stack was still small enough to reason about directly, and confined the one unresolved issue — the camera hardware fault — to the final, independent integration step rather than blocking earlier verification.

IX. EVALUATION

The system was evaluated component-by-component on the physical Raspberry Pi deployment rather than in a simulated environment, so that reported results reflect what was actually exercised in this specific project, not a projection of what should work in principle.

A. Confirmed working end-to-end

- MQTT broker connectivity and topic subscription.
- InfluxDB persistence: adding a user (with a generated PIN), listing active users, and revoking a

user, confirmed to correctly remove the user from subsequent active-user queries.

- QR generation following successful PIN authentication, including rejection of an invalid PIN and of an unknown user.
- QR validation via a manual paste-in test mode added specifically to exercise the authentication-to-access-grant path independently of camera hardware (Section V-D, camera_scanner.py fallback), confirmed to correctly log the event and invoke the (simulated) door-lock and Telegram notification calls.
- The web management UI: adding and removing users from a browser, reflected immediately in the underlying database and in the CLI's own view of the same data.

B. A real bug found and fixed during testing

During testing, a user revoked and then re-added under a different zone was reported as "not found" by the authentication path even though the web UI's list view showed them as active. The cause, discussed in Section IV-B, was that the authentication query read only the first InfluxDB result table rather than scanning all tables for the user and selecting the globally most recent point; since the revoked and the re-added grant used different zone tags, they were held in separate series, and the stale, revoked series happened to be returned first. The fix — scanning every returned series and selecting the point with the latest timestamp — was implemented and verified to resolve the discrepancy between the list view and the authentication path.

C. Not fully validated

Camera-based QR scanning could not be end-to-end validated: rpicam-hello --list-cameras reported no camera detected on the deployment hardware, indicating a hardware or ribbon-cable connection fault rather than a software defect, since the identical decoding path (pyzbar against a captured frame) is exercised successfully by the manual/no-camera test mode used to validate the rest of the pipeline. Physical door-lock actuation and Telegram notification delivery were exercised only through their simulated/unconfigured code paths, since no relay or configured bot token was available during development; the corresponding code was reviewed and unit-level tested but not confirmed against physical hardware or a live Telegram chat.

X. REQUIREMENTS TRACEABILITY

Table II maps each bullet of the original assignment specification to the component that implements it and its verification status, to make the scope of what was built and what was confirmed working explicit at a glance.

Requirement	Status
Access control for an environment (lab)	Done, verified
QR-code authentication	Done, verified
Raspberry Pi camera	Implemented, hardware fault
MQTT as main protocol	Done, verified
Telegram bot for results	Implemented, not live-tested
Access list with add/remove UI	Done, verified (CLI + web)
Optional: door-lock actuation	Implemented, simulated only

TABLE II. REQUIREMENTS TRACEABILITY

XI. LIMITATIONS AND FUTURE WORK

- Diagnose and resolve the camera hardware fault (reseat or replace the CSI ribbon cable, confirm camera enablement) and complete an end-to-end scan-to-unlock test.
- Wire a physical relay and electric strike to validate gpio_lock.py against real hardware rather than its simulated fallback.
- Configure a real Telegram bot token and confirm notification delivery, including the denied-attempt photo path.
- Extend the web UI with an access-log view and live system status, currently available only from the command-line interface.
- Add automated tests for the authentication and QR-validation logic, particularly around the multi-series lookup behavior identified in Section VII-B, to prevent regressions.
- Consider migrating the access-control-list schema to avoid the append-only multi-series subtlety entirely, e.g., by keying solely on user_id and storing zone as a field rather than a tag.

XII. CONCLUSIONS

This report presented a modular, MQTT-centric access control and surveillance system built around short-lived, signed QR codes, evaluated directly on Raspberry Pi hardware rather than only in simulation. Authentication, time-series persistence, the MQTT integration layer, and a web-based management interface were all implemented and confirmed working, including a real concurrency/lookup bug found and fixed during testing. Camera-based scanning and physical door actuation were implemented with safe simulated fallbacks but await a hardware fix and physical wiring, respectively, to be fully validated — a transparent scope for the remaining work rather than a claim of full end-to-end operation.

ACKNOWLEDGMENT

The author thanks Prof. Davide Brunelli for the project specification and guidance throughout the Laboratory of Internet of Things course. The complete source code, including the modules discussed in this report, the MQTT topic definitions, and the deployment configuration, is available at https://github.com/yksop/access_control; the environment file containing secrets (.env) is intentionally excluded from version control, as detailed in Section VII-A, and must be provisioned locally following .env.example before the system can be run.

REFERENCES

[1] ZBar bar code reader, https://github.com/mchehab/zbar, accessed 2026-09-08.

[2] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," in Proc. CRYPTO, 1996, pp. 1–15.

[3] OASIS, "MQTT Version 3.1.1," https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html, 2019, accessed 2026-09-08.

[4] InfluxData, "InfluxDB 2.x documentation," https://docs.influxdata.com/influxdb/v2/, accessed 2026-09-08.

[5] Raspberry Pi Foundation, "Picamera2 library documentation," https://datasheets.raspberrypi.com/camera/picamera2-manual.pdf, accessed 2026-09-08.

[6] Telegram, "Bot API," https://core.telegram.org/bots/api, accessed 2026-09-08.

[7] Eclipse Foundation, "Mosquitto MQTT broker," https://mosquitto.org/, accessed 2026-09-08.

[8] Pallets Projects, "Flask documentation," https://flask.palletsprojects.com/, accessed 2026-09-08.